YENEPOYA UNIVERSITY

# HIGH LEVEL DESIGN (HLD) DOCUMENT
# ON
# Cloud Cost Intelligence For Smart City


**Student Name:** Shuhaib Abdulla

**Registration No:** 23BCAICD101

**Campus ID:** 25296

**Project Code:** PRJN26-108

**Industry Mentor:** Sumit Kumar Shukla

**Guided By:** Ms. Deeksha Shetty

# TABLE OF CONTENTS

**1.** Introduction
- 1.1 Scope of the Document
- 1.2 Intended Audience
- 1.3 System Overview

**2.** System Design
- 2.1 Application Architecture
- 2.2 Process Flow
- 2.3 Information Flow
- 2.4 Components Design
- 2.5 Key Design Considerations
- 2.6 Technology Stack

**3.** Data Design
- 3.1 Data Model
- 3.2 Data Access Mechanism
- 3.3 Data Retention Policies

**4.** Interfaces

**5.** State and Session Management

**6.** Caching Strategy

**7.** Non-Functional Requirements
- 7.1 Performance Aspects
- 7.2 Scalability

**8.** References

# 1. INTRODUCTION

The Cloud Cost Intelligence Platform for Smart City is an analytics-driven application designed to process, visualise, and report cloud billing data generated by smart city infrastructure services. As municipalities increasingly migrate their operational systems to cloud environments, the challenge of monitoring and optimising cloud expenditure across diverse departments becomes critical. This platform addresses that challenge through automated data ingestion, multi-dimensional cost analysis, and exportable reporting.

## 1.1 Scope of the Document

This High-Level Design (HLD) document outlines the macro-level architecture, data design, component interactions, and interface specifications of the Cloud Cost Intelligence Platform. It defines the structural boundaries of the system and describes how the various analytical tiers communicate with each other. Granular programmatic logic, individual function signatures, and detailed algorithmic steps are intentionally reserved for the accompanying Low-Level Design (LLD) document.

## 1.2 Intended Audience

- **Academic Evaluators and Instructors:** To assess the architectural soundness and design methodology of the project (PRJN26-108).
- **Industry Mentors:** To review the system's macro-architecture and ensure its alignment with real-world cloud cost optimisation standards.
- **Software Developers:** To serve as the primary implementation blueprint for building the data pipeline, visualisation engine, and reporting modules.
- **Data Analysts:** To understand how raw billing records are transformed into actionable insights across service types, months, and departments.

## 1.3 System Overview

The platform ingests a structured CSV file containing Smart City cloud billing records — covering five service types (Compute, Storage, Networking, Database, AI/ML Services) across five departments (Traffic Management, Public Safety, Smart Lighting, Waste Management, Water Supply) over a twelve-month period. The engine performs grouping, aggregation, and rolling analytics before generating pie charts, bar charts, and a comprehensive Excel summary report. The system is built entirely in Python using Pandas, Matplotlib, and OpenPyXL, requiring no external API dependencies or cloud services to operate.

# 2. SYSTEM DESIGN

## 2.1 Application Architecture

The platform is structured as a sequential, three-tier data processing pipeline that cleanly separates ingestion, transformation, and output concerns.

| Tier | Component | Responsibility |
|---|---|---|
| Ingestion Tier | CSV Loader (Pandas) | Reads and validates the raw smart_city_billing.csv file into a structured DataFrame. |
| Processing Tier | Analytics Engine (Pandas) | Performs grouping, aggregation, pivot operations, and rolling statistics on the billing data. |
| Visualisation Tier | Chart Generator (Matplotlib) | Produces pie charts and bar charts for service-type, monthly, and departmental cost views. |
| Export Tier | Report Writer (OpenPyXL) | Serialises summary tables and analysis results into a formatted multi-sheet Excel workbook. |

## 2.2 Process Flow

The high-level operational flow follows these sequential steps:

- **Data Ingestion:** The billing CSV is loaded into a Pandas DataFrame. Column types are validated and display settings are configured.
- **Exploratory Analysis:** Dataset dimensions, column metadata, basic statistics, and null-value counts are computed and logged.
- **Service-Type Aggregation:** Records are grouped by Service_Type and total Cost_USD is summed and rank-ordered.
- **Monthly Aggregation:** Records are grouped by Month, reindexed to calendar order, and summed for temporal trend analysis.
- **Departmental Aggregation:** Records are grouped by Department to reveal which city operations consume the most cloud budget.
- **Pivot Analysis:** A Month × Service_Type pivot table is constructed to enable stacked bar visualisation of multi-dimensional spend.
- **Visualisation:** Pie and bar charts are rendered and saved as PNG artefacts.
- **Excel Export:** All aggregated summaries are written into a structured, multi-sheet Excel workbook for stakeholder distribution.

## 2.3 Information Flow

Data flows linearly through the pipeline. The raw CSV file is the sole external input. All intermediate transformations (grouped Series, pivot DataFrames, rolling statistics) exist entirely in memory. The final outputs — PNG chart files and an Excel workbook — are written to the local filesystem. There is no network communication, database connection, or external API dependency involved at any stage of the

pipeline.

## 2.4 Components Design

The architecture ensures component independence. Each analytical stage can be tested, modified, or replaced without affecting other stages. For example, the visualisation tier can be swapped from Matplotlib to Plotly, and the export tier can be changed to produce PDF reports instead of Excel — all without altering the core analytics logic.

## 2.5 Key Design Considerations

- **Month Order Preservation:** Python's groupby operation returns months in alphabetical order by default. The system explicitly reindexes aggregated monthly data against a defined calendar-order list to preserve chronological integrity in all charts and reports.
- **Extensibility:** The platform's tabular input contract (CSV with Invoice_ID, Month, Service_Type, Department, Usage_Hours, Cost_USD) is generic enough to accommodate future smart city data expansions such as satellite connectivity costs, IoT sensor billing, or multi-region cloud deployments.
- **Reproducibility:** All warning suppression, float display formatting, and DPI settings are configured at initialisation to ensure consistent output across different execution environments.

## 2.6 Technology Stack

| Library | Version | Purpose |
|---|---|---|
| Python | 3.x | Core execution environment |
| Pandas | ≥ 1.3 | Data ingestion, grouping, aggregation, and pivot operations |
| Matplotlib | ≥ 3.4 | Pie chart and bar chart generation |
| OpenPyXL | ≥ 3.0 | Multi-sheet Excel report export |
| Warnings | stdlib | Suppression of non-critical runtime warnings |

# 3. DATA DESIGN

## 3.1 Data Model

The system's persistent input model relies on a single flat CSV file: **smart_city_billing.csv**. This file stores individual cloud service invoice records for all smart city departments across a 12-month period. The dataset contains 60 records (5 services × 5 departments × 12 months / 5).

| Field | Type | Description |
|---|---|---|
| Invoice_ID | String | Unique invoice identifier (e.g., INV-001) |
| Month | String | Calendar month name (January through December) |
| Service_Type | String | Cloud service category: Compute, Storage, Networking, Database, AI/ML Services |
| Department | String | Smart city department: Traffic Mgmt, Public Safety, Smart Lighting, Waste Mgmt, Water Supply |
| Usage_Hours | Integer | Total hours the cloud service was consumed during the billing period |
| Cost_USD | Float | Total monetary cost in US Dollars for the billing record |

## 3.2 Data Access Mechanism

During pipeline execution, the entire CSV file is loaded into a single Pandas DataFrame via **pd.read_csv()**. All subsequent aggregations, groupings, and pivots operate on this in-memory DataFrame. There is no lazy loading, no chunked reading, and no database query layer — the dataset size (60 records) makes full in-memory processing optimal and appropriate.

## 3.3 Data Retention Policies

Given the academic scope of this project, data retention is managed locally via the filesystem. The input CSV and all output artefacts (PNG charts and Excel report) are stored in the project working directory. No automated archival schedule, cloud sync, or database backup mechanism is implemented. All intermediate DataFrames exist exclusively in RAM and are released when the Python session terminates.

# 4. INTERFACES

The system provides outputs through two primary interface channels depending on the analysis requirement.

## 4.1 Analytical Visualisation Interface

The platform produces four distinct chart visualisations:

| Visualisation | Chart Type | Data Dimensions |
|---|---|---|
| Cost by Service Type | Exploded Pie Chart | Service_Type vs. Sum(Cost_USD) |
| Cost by Month | Pie Chart w/ $ Labels | Month vs. Sum(Cost_USD) |
| Monthly Spend per Service | Stacked Bar Chart | Month × Service_Type vs. Sum(Cost_USD) |
| Spend by Department | Horizontal Bar Chart | Department vs. Sum(Cost_USD) |

## 4.2 Excel Report Interface

The export tier writes a structured Excel workbook containing three summary sheets: a service-type cost breakdown, a monthly cost breakdown, and a department-level cost breakdown. Each sheet contains the aggregated totals directly derived from the in-memory Pandas groupby results, formatted for stakeholder review and further manual analysis.

## 4.3 Console / Logging Interface

Throughout execution, the pipeline emits structured print-based log messages confirming each processing step — from library import confirmation through dataset loading, column inspection, and per-stage cost summaries. These console outputs provide real-time progress visibility without requiring an external logging framework.

## 5. STATE AND SESSION MANAGEMENT

The platform operates in a single-session, stateless execution model. The pipeline is executed top-to-bottom within a Jupyter Notebook environment. All intermediate state — loaded DataFrames, grouped Series, pivot tables, and chart figure objects — exists purely in the notebook kernel's memory for the duration of the session.

There is no persistent session state, no user login context, and no cross-run state accumulation. Each execution of the notebook from Cell 1 produces a fully deterministic, reproducible result from the same input CSV. This design eliminates state-related bugs and simplifies debugging, testing, and academic reproduction.

## 6. CACHING STRATEGY

The system employs an implicit in-memory caching strategy through Pandas DataFrame variable reuse. Once the raw CSV is loaded into the master DataFrame **df**, all subsequent analytical transformations — service groupings, monthly groupings, department groupings, and pivot tables — are derived from this single shared object without re-reading the file.

Computed aggregated Series (e.g., **service_cost**, **monthly_cost**, **dept_cost**) are assigned to named variables and reused across multiple downstream operations (charting and export), eliminating redundant computation. External distributed caching systems (e.g., Redis, Memcached) are not applicable at the current scale and academic scope of the project.

# 7. NON-FUNCTIONAL REQUIREMENTS

## 7.1 Performance Aspects

The platform is optimised for rapid processing of small-to-medium billing datasets. On a standard development machine, the complete pipeline — from CSV ingestion through chart generation and Excel export — executes in under five seconds for the current 60-record dataset. Pandas' vectorised operations ensure that performance scales linearly with dataset size without requiring algorithmic restructuring for datasets up to several hundred thousand records.

Chart rendering DPI is configured at 100 for screen quality and elevated to 120 for saved PNG artefacts, balancing visual clarity with file size.

## 7.2 Scalability

- **Data Volume:** The CSV-based ingestion model can scale to larger billing datasets without code changes. Pandas chunked reading can be introduced if memory constraints arise at scale.
- **Service Extension:** Additional cloud service types or departments can be added to the CSV without modifying any analytical or charting logic, as all groupings are dynamically derived.
- **Output Extension:** The export tier can be extended to produce additional formats (PDF reports, JSON summaries, database insertions) by appending new output cells to the notebook pipeline.
- **Visualisation Extension:** Interactive dashboard frameworks (Streamlit, Dash) can wrap the existing Pandas analytics layer to provide browser-based exploration without replacing the core engine.

# 8. REFERENCES

**[1]** Low-Level Design (LLD): Cloud Cost Intelligence Platform for Smart City (PRJN26-108) — Contains detailed function-level logic, data transformation formulas, and module specifications.

**[2]** McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference.*

**[3]** Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering, 9(3), 90–95.*

**[4]** OpenPyXL Documentation: https://openpyxl.readthedocs.io — Excel file read/write operations in Python.

**[5]** Pandas Documentation: https://pandas.pydata.org/docs — DataFrame groupby, pivot_table, and reindex API references.

**[6]** Smart City Cloud Billing Dataset: smart_city_billing.csv — Internal project dataset covering 5 service types, 5 departments, and 12 months of cloud expenditure records.